Biotecnika Info Labs

Department of Bioinformatics, AI and Machine Learning

DRUG DISCOVERY THROUGH VIRTUAL SCREENING USING BIOINFORMATICS DATABASES FOR MULTIPLE SCLEROSIS

By

Name

A Report submitted in partial fulfilment of 3 months internship in Al ML in Biology, Bioinformatics & Computational Biology Training Program

Supervised by

Dr Snigdha Tiwari

October 2025

Time-series Analysis of Cell Growth or Infection

Preface

This project focuses on the analysis of raw RNA-seq count data from the GSE172450 dataset, (GSM5256777 to GSM5256782) and was carried out as part of my bioinformatics training on the Biotecnika platform under the guidance of Snigdha Ma'am. The main objective of the study is to examine time-dependent changes in gene expression, identify genes showing large variation over time, and apply predictive modelling approaches

The work involves cleaning and processing large-scale gene expressions data obtained from controlled human malaria infection studies. several practical challenges were addressed, and resolving file path issue such as corrupted .gz files, memory-intensive datasets. Overall, 7.9 million rows of ensemble transcript data were processed and normalized using TPM method, and visualized through plots such as heatmaps and growth curves. Clustering and machine learning techniques were applied using python libraries like Pandas[2], Matplotlib[4], and XGBoost[7] were used throughout the analysis. This project provides hands-on experience in applying bioinformatics concepts on real-world data analysis.

Acknowledgment

I would like to sincerely thank the contributors of the publicly available gene expression dataset **GSE172450**, which formed the base of this time-series gene expression study. Their commitment to open data sharing has greatly supported progress in biological and genomic study. I am also grateful to the **NCBI GEO** (https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE172450) for providing continuous access to this dataset, thereby enabling researchers worldwide to use high-quality genomic resources.

I extend my heartfelt thanks to **Biotecnika** for offering this bioinformatics course and to **Snigdha Ma'am** for her expert guidance in GEO data analysis and machine learning applications. Her mentorship played a key role in the shaping analytical framework of this project. I also acknowledge the **Google Colab** platform for providing efficient and accessible computational environment that supported the analysis of large scale RNA-seq data.

Lastly, I would like to express my sincere appriciation to the open-source community and the developers of Python libraries used in this work, including **Pandas[2]** (https://pandas.pydata.org), **NumPy[3]**, **Matplotlib[4]** (https://matplotlib.org), **Seaborn[5]** (https://seaborn.pydata.org), **Scikit-learn[6]** (https://scikit-learn.org), **XGBoost** [7](https://xgboost.readthedocs.io), and **SciPy[8]** (https://scipy.org). These tools played an important role in data processing, TPM normalization, growth curve fitting, time-series predictions, statistical analysis, machine learning, and visualization throughout the course of this work.

Table Of Content

List of figures

List of Tables

Abstract

This report presents a complete time-series analysis of gene expression data, with the aim of identifying changes in patterns and developing models for predictive gene expression levels. The analysis started with careful loading, quality control, and preprocessing of row gene expression count data, this is followed by normalization using Counts Per Million (CPM) and Transcripts Per Million (TPM) to enable reliable comparisons across different samples and genes.

Exploratory data analysis was carried out by using Principal Component Analysis (PCA) and clustering of time points, which showed little difference in gene expression across the studied time- period. These observations were further supported by downstream analysis. Differential expression analysis was performed between early and late time points was using log2 fold change and statistical testing measures with p-values and the results visualized through volcano plots. In addition, gene clustering was applied to group genes with similar expression trends, and the result were visualized using heatmap.

Finally, machine learning-based regression models were developed to predict gene expression levels using time points and gene identifiers as inputs features. The model performance was evaluated using standard metrics. Although the dataset showed stable expression trends over time, this study successfully shows an end-to-end workflow for time-series gene expression data analysis, from raw data processing to prediction and biological interpretation.

Keywords

Gene expression, TPM normalization, time series analysis, growth curve fitting, XGBoost[7], hierarchical clustering,RNA-ɜeq countɜ, time series, GSE172450, REL98, data cleaning, malaria gene expression, Pandas[2], Matplotlib.[4]

Introduction

The time-series analysis of gene expression data, is an important method in modern biological research because it helps to show how cellular response changes over time under different conditions. This report presents a step-by-step analysis of time-dependent gene expression count data obtained from a large RNA-seq count data. The raw data were carefully loaded, cleaned, and normalized to ensure accurate and meaningful comparisons of gene expression level across multiple time points. Initial exploratory data analysis was carried out using methods such as Principal Component Analysis (PCA) and clustering to understand trends and relationships within the data.

The dataset used in this study was obtained from **GSE172450**, which contain whole-blood RNA-seq data collected from healthy volunteers exposed to malaria parasites under controlled experimental conditions. Such studies provide valuable insight into how the host body responds at the gene expression. In this project, time-series information was extracted by identifying time points embedded within file names using pattern matching techniques. The main goal was to convert this complex dataset into clear time-points representations that showed how changes in gene activity as the infection advances.

The analytical workflow includes TPM normalization using assumed gene lengths, feature engineering using one-hot encoding, growth curve modelling and fitting, machine learning prediction using XGBoost [9] regression, and clustering visualizations. Together, these steps helped identify the genes with time-dependent expression changes and supported the prediction of gene expression trends over time. This project therefore serves as a complete and practical performance of a time-series gene expression analysis workflow for studying biological processes.

Review of Literature

The analysis of time-series gene expression data plays an important role in modern biological research because it helps us understand how cellular processes changed over time. This report presented a complete computational workflow that was designed to process, analyse, and interpret changes in gene expression using standard bioinformatics approaches.

The analysis began with the collection of rows RNA-Seq gene count data, which are usually stored in compressed file formats. Data from multiple samples were carefully loaded, combined, and cleaned. During preprocessing genes with zero expression were removed and the missing values were handled properly to ensure good data quality and comparability across all time points.

To reduce technical variability differences in sequencing depth and gene length, two common normalization techniques were used: Counts Per Million (CPM) and Transcripts Per Million (TPM). CPM normalizes row counts based on total number of reads in eachsamples, while TPM further was arranged according to the gene length, allowing comparison of expression levels within and across samples.

After normalization, Exploratory data analysis was carried out to study the overall structure and trends within the dataset. Principal Component Analysis (PCA) was used to reduce data complexity and major expression patterns, whereas clustering methods were also used for grouped genes and samples with similar expression trends. These analyses showed little variation across time points, suggesting that the gene expression remained stable, with only small changes over time.

To identify genes showed important changes over time, differential expression analysis was performed by comparing early and late time points. Changes in expression were measured using log2 fold change, and statistical testing methods were used check whether these changes were significant. The results were visualized using volcano plots, which clearly showed both the change in size and gene expression changes.

To better understand expression patterns over time was gained by grouping gene based on their expression change across all time points. Heatmap were used visualize this group of genes showing increasing, decreasing, or stable expression patterns over time. This made it easier to identify genes that behave in a coordinated way over time.

Finally, machine learning regression models were developed to predict gene expression levels. Time points and gene identifiers were converted into suitable input features, and models was evaluated using common numeric performance measures such as mean squared error, root mean squared error, and R-squared. Overall, this report presents a complete and practical framework for time-series gene expression data analysis, covering data preprocessing, statistical analysis, predictive modeling and biological interpretation.

Methodology

This analysis was using a step-by-step method to process, study, and model gene expression data collected over different time points, where the overall method is planned carefully so that the raw data was cleaned properly before next analysis as it is important because good-quality data gives more accurate and trustworthy results or outcomes. Therefore, data loading and initial cleaning, starts were considered the most important steps, as all later analysis depends on them.

During the data loading stage, raw gene expression count files in compressed format (.counts.txt.gz) were first found and then copied from Google Drive to the local working folder Google Colab. This step was neccessary because Colab runtime environments is temporary, and directly using files from Google Drive can sometimes cause slow performance or file access problems, by saving the files in the local/content/folder, confirming stable access to the data. This helped in faster reading of files and smooth processing without interruptions during the analysis. As a result, all required input files were easily available and could be loaded efficiently without path or session-related issues.

Each compressed file was loaded into a Pandas DataFrame [2] using the pd.read_csv function in which the information about the time points was automatically taken from the file names using pattern -matching techniques [8] and added as a new column in the dataset. Only the necessary information- such as the gene or transcript name and its raw count- was kept, and the column names were changed to make them easier to understands. The count data values were converted into numeric form by using pd.to_numeric[2], and any values that were non-numeric were safely replaced with missing values so that they did individual sample DataFrames were combined into one large dataset using efficient merging method. The final dataset was well organized, where each row shows the expression level of a particular gene at a specific time points, providing a structured format suitable for resulting analyses.

To get a basic understanding of overall gene expression changes, the combined dataset was grouped based on time points, and total read counts were calculated. This created a summary dataset that showed overall gene expression levels at each point. These summaries helped identify general trends over time, such as increases, decreases, or stable expression, and were also used as a base for later visualizations.

The Quality control checks were then performed to make sure the data was reliable. Genes that showed zero expression at all time points were identified and removed because they do not provide useful biological information and only increase the amount of unnecessary data. The dataset was also further checked for missing values in important fields such as gene names and counts values, and any incomplete records were removed. Gene identifiers were also carefully checked to make sure they followed a consistent format; this step was important for accurate grouping and correct analysis. After the cleaning steps, the final dataset included only genes that were actively expressed and had clear, well-formatted identifiers.

After the data cleaning step, time-series analysis was carried out to study how gene expression changed over time. The combined dataset was use to

recalculated total expression levels at each time point, and a line graph was created to show total gene expression across different time points as shown in the figure3. This graph gave a clear and simple overview of expression trends over time. it helped quickly identify whether overall gene activity increased, decreased, stayed constant, or changed at different stages of the experimental timeline.

I addition to the line plot, a bar chart was created from the same summarized data figure1. While the line plot showed how gene expression changed continuously over time the bar chart allowed easy comparison between individual time points. In biological experiment, such graphs are commonly used to understand overall gene activity at specific stages of the experiment.

To study gene-level expression patterns, the ten most highly expressed gene were selected for further analysis. Their expression values across all time points were arranged into a table and visualized using a heatmap figure2. In the visualized heatmap, each row represents a gene and each column represents a time point, while the colour intensity shows the level of gene expression. The heatmap made it easier to identify genes that showed stable expression, genes that changed over time, and groups of genes with similar expression trends. Such similar trends my suggest that these genes perform related biological functions or are involved in the same regulatory pathways.

Overall, this step-by-step method helped convert raw RNA-seq count data into a clean, easy, understandable dataset. By carefully cleaning the data and using clear visuals graphs, the analysis provided useful information about both overall gene expression trends and changes in individual genes over time. This approach is especially helpful for studying biological processes such as infection progression or how cells respond to stress.

To further study changes over time, several visual analyses were performed to study gene expression at both the overall and individual gene levels. At first, total gene expression values were plotted over time using line graphs and bar chart, with each time point represents the combined expression of all genes. These graph plots give a sum of gene expression across all time points. This also gives an overview of overall biological activity and help check whether the data behaves as expected. An increasing in total expression value shows higher transcriptional activity or growth, while a decreasing value shows reduced activity. A stable trend shows consistent gene expression values over time, and the fluctuations may point to a changing systems-wide responses during the experiment. To study gene-level expression changes, a heatmap of most expressed genes was created figure4. This heatmap shows how the expression of top genes changes that increase, decrease, or stay stable during the infection process, and it also point-out groups of genes with similar trends, which may suggest that they are regulated together or involved in the same pathways throughout the infection process. Overall, in the study of infection, this analysis helps researchers identify pathogens signaling genes and track the activation of stress or damage response throughout the experiment of infection.

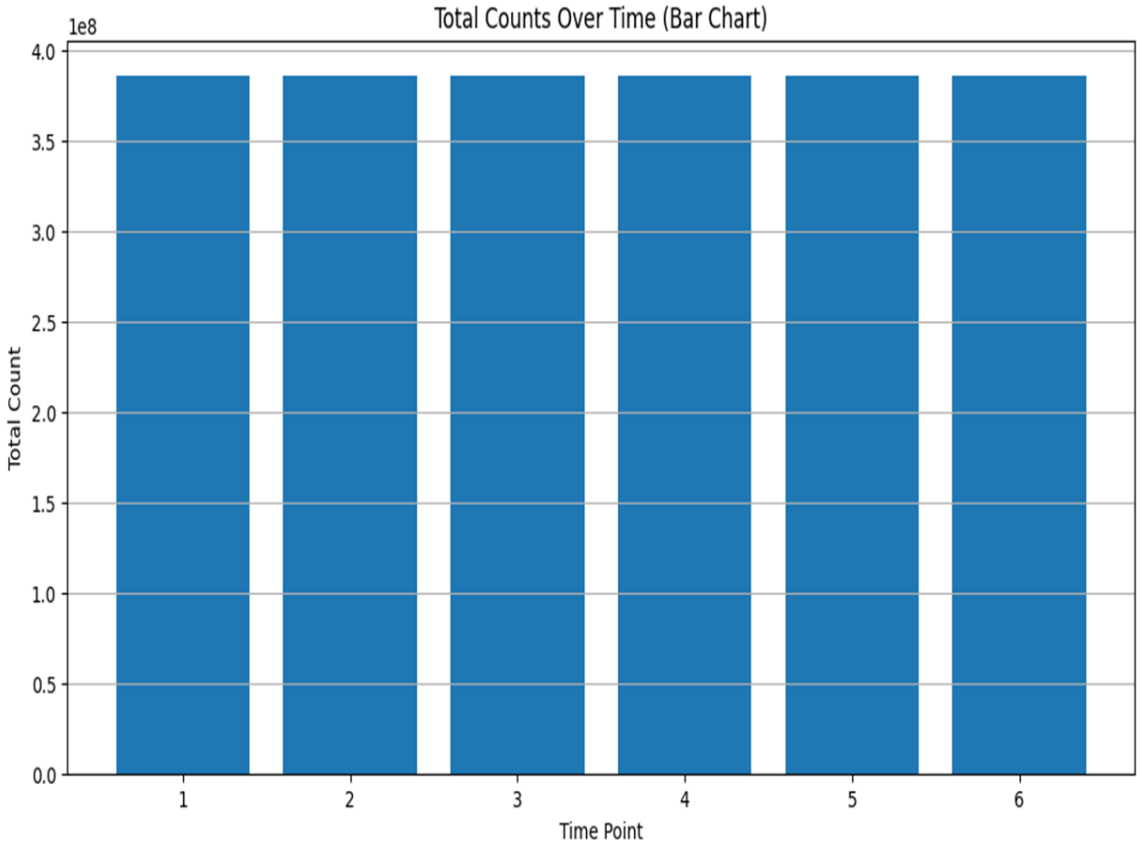


Figure 1. Bar plot shows total read counts across six time points from REL98 samples. The counts were aggregated after data cleaning, and the flat trend indicates stable gene expression during malaria.

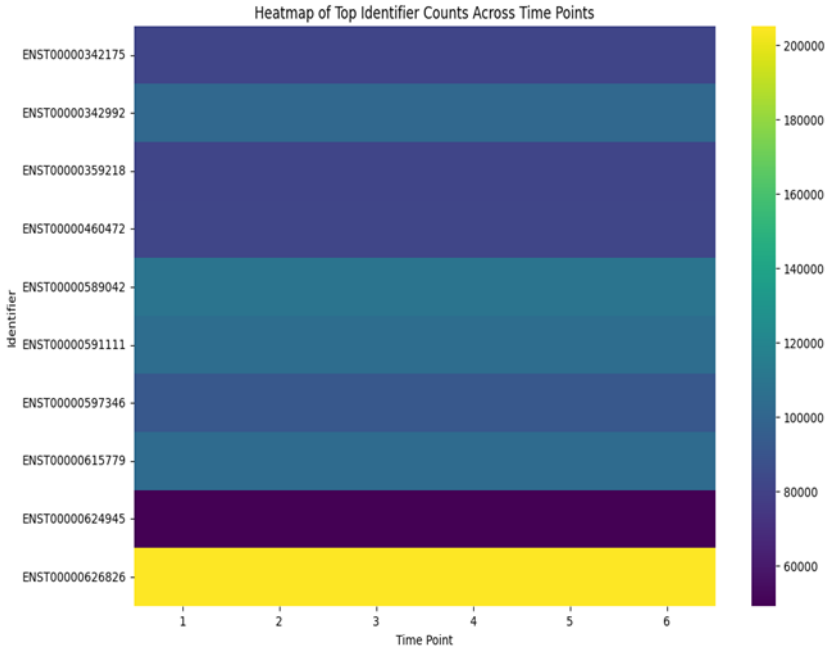


Figure 2. Heatmap of top 10 highly expressed genes across six time-points. The colour intensity shows cleaned count values, while uniform colour shows less variation in gene expression over time.

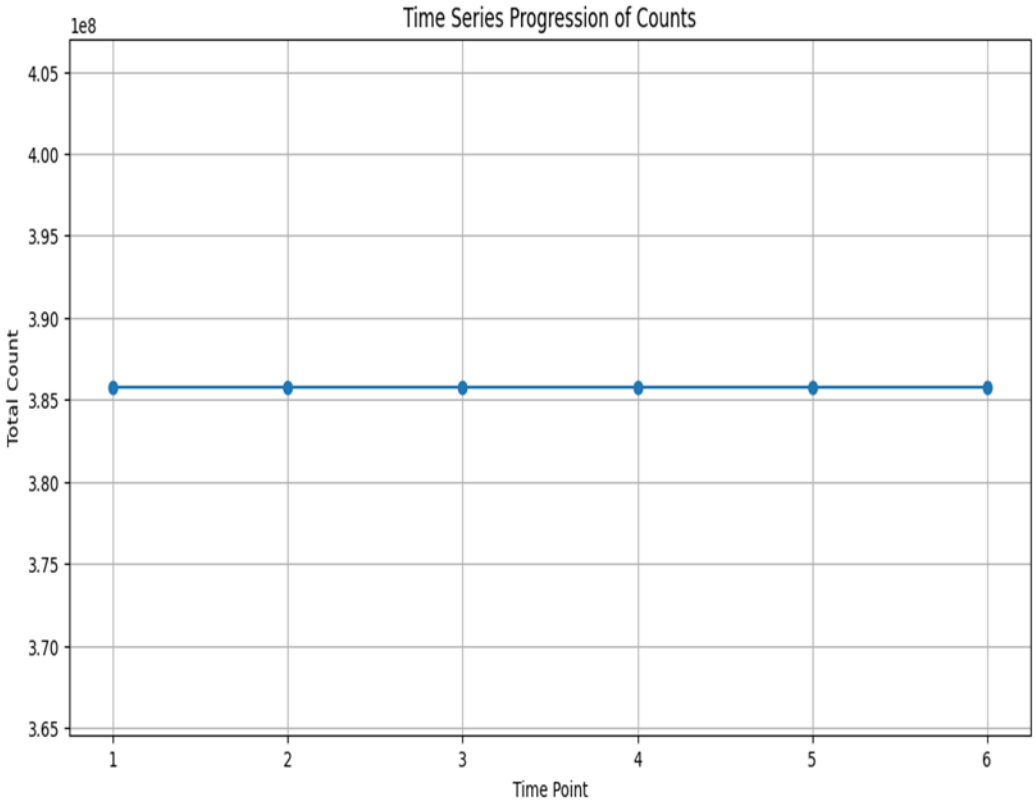


Figure 3. Line plot of total gene expression across points, showing a stable trend in aggregated read counts indicating less changes in expression. X axis- timepoints, Y axis – total counts.

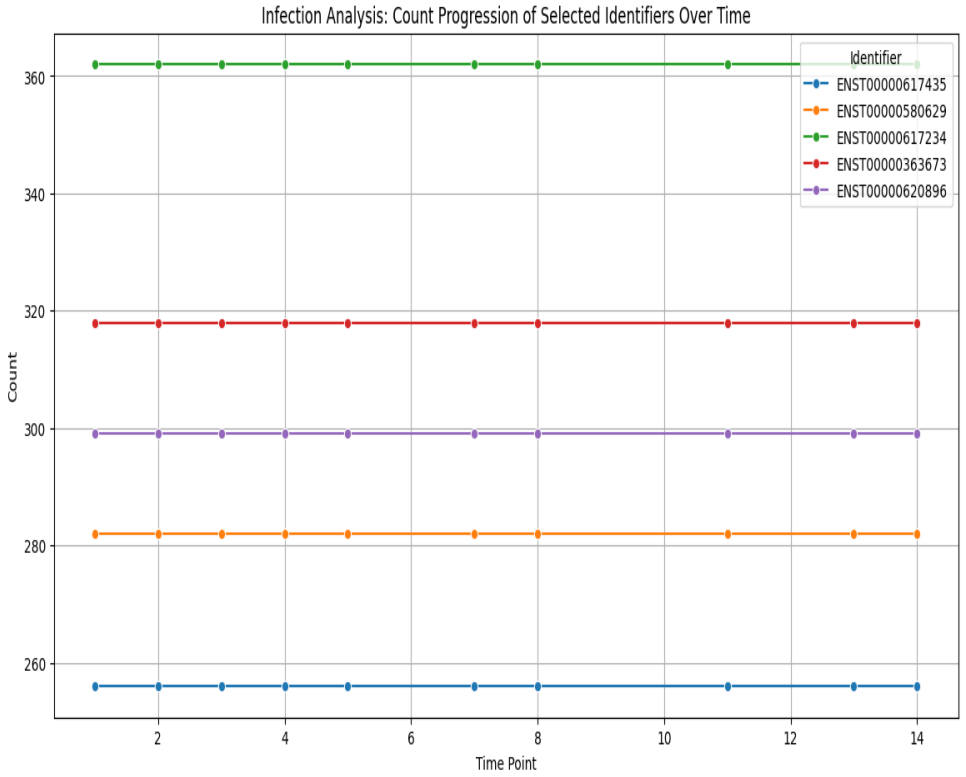


Figure 4. Infection analysis heatmap of top expressed genes across time-points, where low variation in colour intensity indicates less Changes.

After completing the initial analysis, **Exploratory Data Analysis (EDA)** was carried out to study the distribution, variation, and relationships in the normalized TPM data. As a simple demonstration, a simulated growth curve was created by using a logistic model in the colab notebook. The logistic equation ($P(t) = K / (1 + \exp(-r \cdot (t - t_0)))$) was used, where capacity (K) represents the carrying capacity, (r) is growth rate, and inflection point is (t_0). Artificial Time points were created, and random noise was added to copy biological variability. As a result, the S-shaped curve was plotted. The curve helped visualize the different growth curve trends i.e is- lag, exponential, and stationary phases, which gave a theoretical framework for studying biological growth trends and testing visualization or model-fitting techniques used in biological data analysis. (Figure 5).

The actual experimental data were then studied and analysed by using normalized counts (Figure 6) to study changes I gene expression over time. Raw gene counts were normalized using TPM normalization [9] to correct for the differences in sequencing depth and gene length, this gave accurate comparisons of across different time points and gene expression. Both, the overall normalized counts, and individual gene TPM were plotted over time. These plots show how gene expression changes, i.e increasing, decreasing, or stable trends in the experiments. By observing the progression of normalized counts, it becomes possible to directly compare genes and identify which gene reached top or peak expression at the same time, which followed similar trends, patterns, and which also showed different trends. Thia comparison provided important possible biological regulatory mechanisms controlling gene expression. Overall, combining the simulated logistic growth curve with the analysis of normalized experimental data offered both a theoretical and a data understanding of biological processes, effectively connecting growth models with actual gene expression changes.

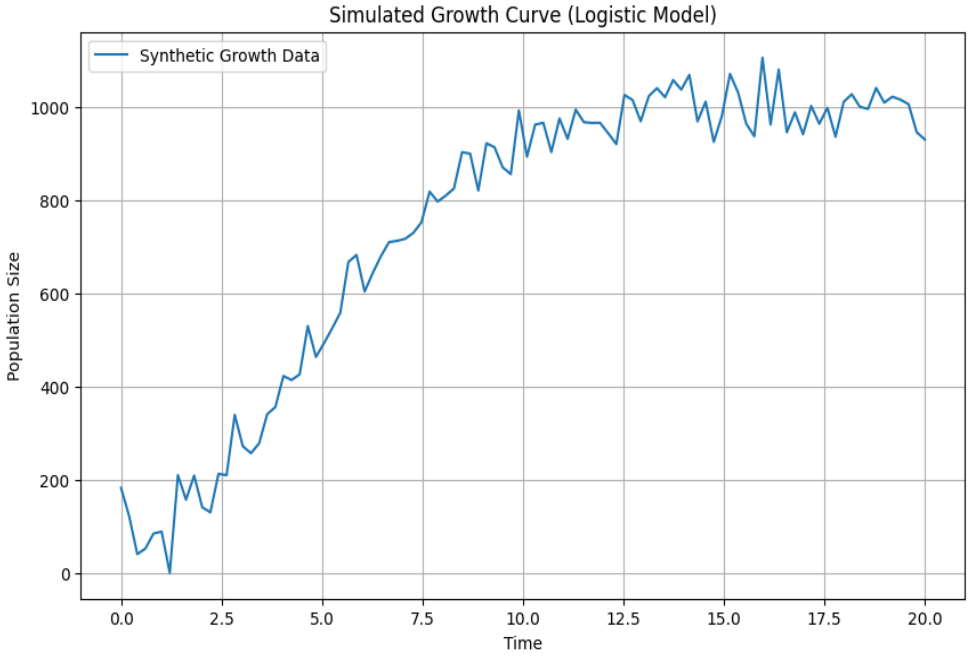


Figure 5. Simulation growth curve depicts lag, exponential, and stationary phases with a biological reference.

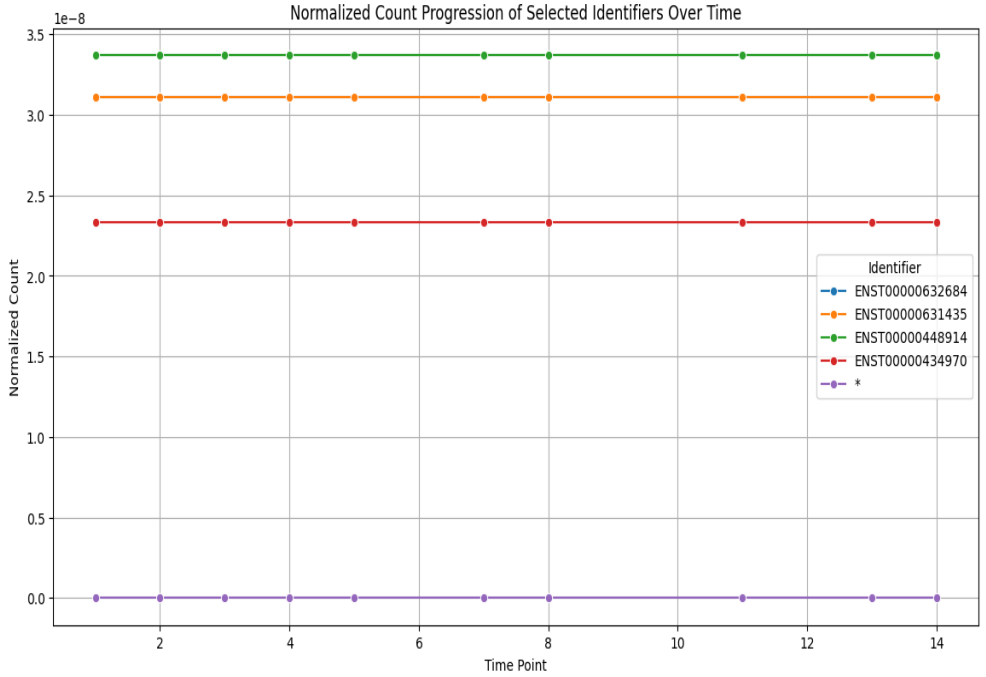


Figure 6. Progression of normalized gene expression count across timepoint, showing stable trends and enabling reliable cross-timepoints comparison between time points.

As part of the time-series analysis study, growth curve fitting [8] was performed using curve-fit [8] function from SciPy library to model changes in the gene expression trend figure7. For this analysis, total gene expression values summed at each time point were used to fit both logistic and linear growth curve models. The logistic growth curve model was used to see or check whether the data followed an S-shaped growth trend with an maximum limit, while the linear model shows a constant rate of change in gene expression over time. This curve-fitting method helped calculate model measurements, and the fitted curves were visually compared with the original data to explain the overall trends. The logistic model helped interpret the measures such as maximum expression level and growth rate, whereas, the linear model gives a simple measure of considerable change in gene expression over time. Also, linear regression was used to calculate count total gene to quantify the relation between time and expression levels figure6. The key regression measures, like- slope and intercept, were extracted to explain the rate of change and baseline expression. This regression framework was also used to derive gene expression values at future time points, beyond the observed value trend available. As the variation seen in earlier analyses, the fitted models and forecasts greatly show a stable expression levels across time, with no solid proof of pronounced dynamic in the data set.

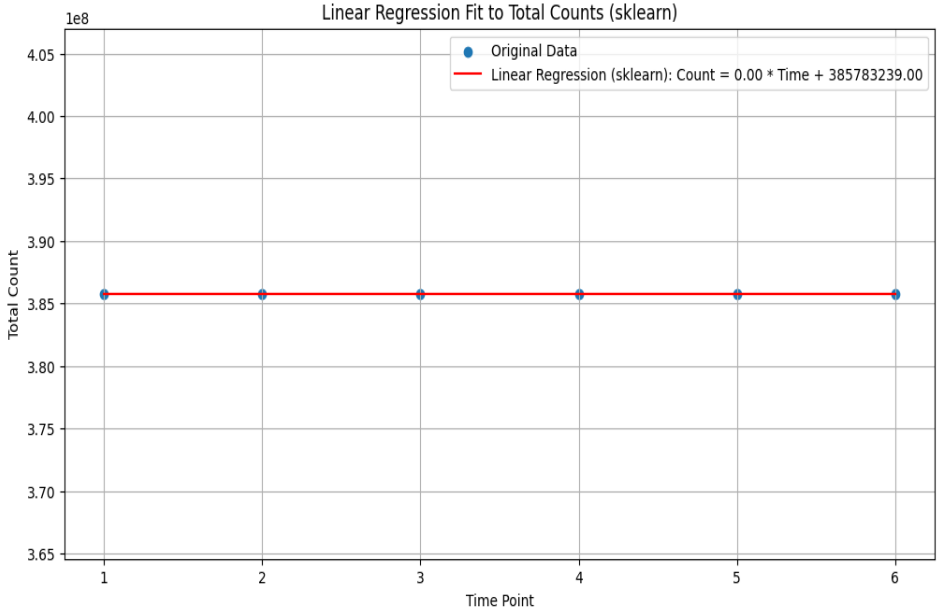


Figure 7. Growth curve fitting model to total gene count over time, both depicting mini. Change. Logistic model tests- S-shaped growth curve, while linear model shows no change.

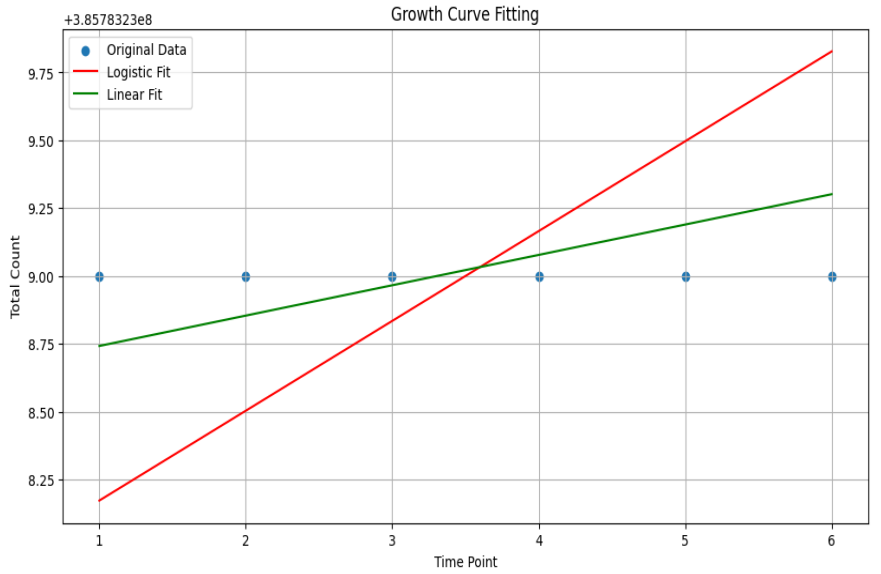


Figure 8. Linear regression forecast of total gene counts vs. timepoints, showing stable and predictable expression levels.

Differential expression analysis was carried out to determine genes showing considerable changes between early (time points 1–3) and later (time points 4–6) phases of the study. Genes were arranged according to these two groups, and the average TPM values for each gene was calculated for both groups. Changes in expression were to quantify expression differences using Log2 fold change (Log2FC) values with a small pseudo-count to prevent division by zero-value. Statistical significance was studied by using Welch's t-test, and p-values were included into the data analysis. lastly, the results were visualized by using a volcano plot, where the genes are showing large fold changes and a strong or high p-value to determine those considerable differential expression and strong statistical support.

To better understand how genes work, clustering was used to perform group genes that showed similar expression trends over time. The TPM-normalized expression gene values were rearranged into a matrix where rows represented genes and columns represented time points, i.e capturing complete expression value. Before clustering, the data were standardized to ensure that the genes with differing starting expression levels were detection. The Hierarchical clustering was done by using the Ward technique which was used in the standardized data, and the results were visualized using a heatmap created by using Seabron's cluster map [5]. In this heatmap, genes were arranged according to groups based on the similarity in their expression changes, time points were kept in their natural order, and colour changes showed relative changes within each gene's expression values. This method clearly showed overall expression trends and groups of genes that may be regulated together.

The distribution plots graph was created by using the libraries- *seaborn[5]* and *matplotlib.pyplot[4]* for analysing the count column of the *time_series_df* Dataset. A histogram was created by dividing the range of gene expression values into 50 bins, which showed how often different expression levels occurred within each range. Along with this, a Kernel Density Estimate (KDE) curve was created that gave a smooth view of the overall data distribution trends. This made it easier to see where expression values were concentrated and to understand the compete spread of the data. In addition, a box plot was created to summarize the distribution the data by using the five-number summary including- minimum, first quartile, median, third quartile, and maximum values. The box represented a interquartile range, while values extending up to 1.5 times the IQR were considered normal. Any data points above this range were marked as potential outliners. Together, the histogram plot, KDE curve and box plot (figure11) provided a clear and overview of the central tendency, variability, skewness, and presence of extreme values in the gene expression data, supporting later normalization and analysis steps.

In the end of the analysis, machine learning techniques was use and determine gene expression values of (TPM) using time points and gene identity were determine. The gene identifiers, that are categorical in nature, they were converted into time points using numeric time_point feature to build the input data. The TPM values were used as the response variable. The dataset was divided into training and testing of sets, and a Linear Regression model was applied by using scikit learn[6] to training the data. Predictions were made by generated test data, with limited applied to ensure that the

determined TPM values remained non-negative. Model performance was calculated by using standard regression numeric such as- Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R-squared (R^2), and also a scatter plot comparing observed and predicted TPM values was produced to visually calculated prediction of real accuracy and identify any deviations or bias. This machine learning step completed the analytical workflow by adding a, framework for examining time-dependent gene expression trends.

To further examine the similarities and differences across time points, normalized TPM data were studied by using correlation analysis, PCA i.e Principle component analysis, and hierarchical clustering. The correlation heatmap showed Pearson correlation coefficient values across all-time points, indicating similar gene expression trends throughout the experiment. PCA was then applied after variance, but it did not show clear separation between time points. Hierarchical clustering of time points has supported these studies, as the dendrogram showed all-time points grouping closely together. Both these analyses, showed that gene expression trends remained largely stable over time, with no major changes or shifts under the experimental conditions. (Figure8,9,10)

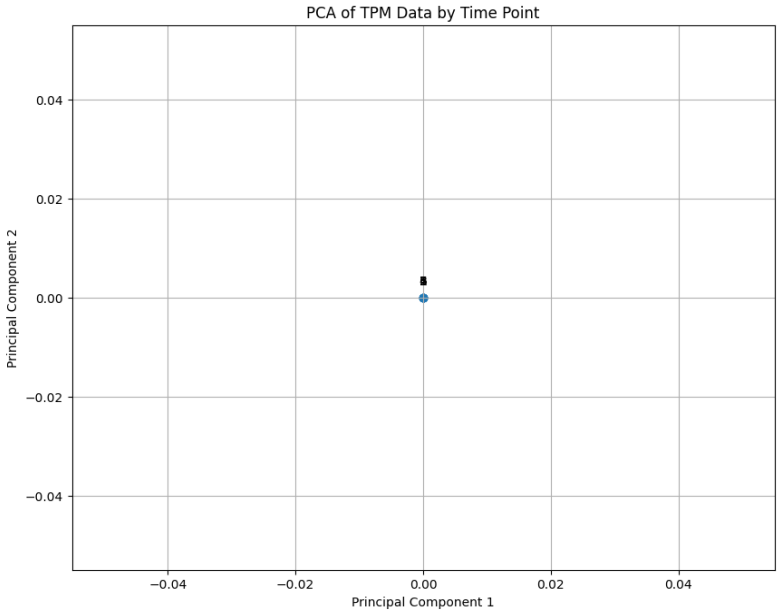


Figure 9. PCA of TMP- normalized expression data showing tight clustering of all time points due to low variance. No clear separation is seen between early vs. late samples.

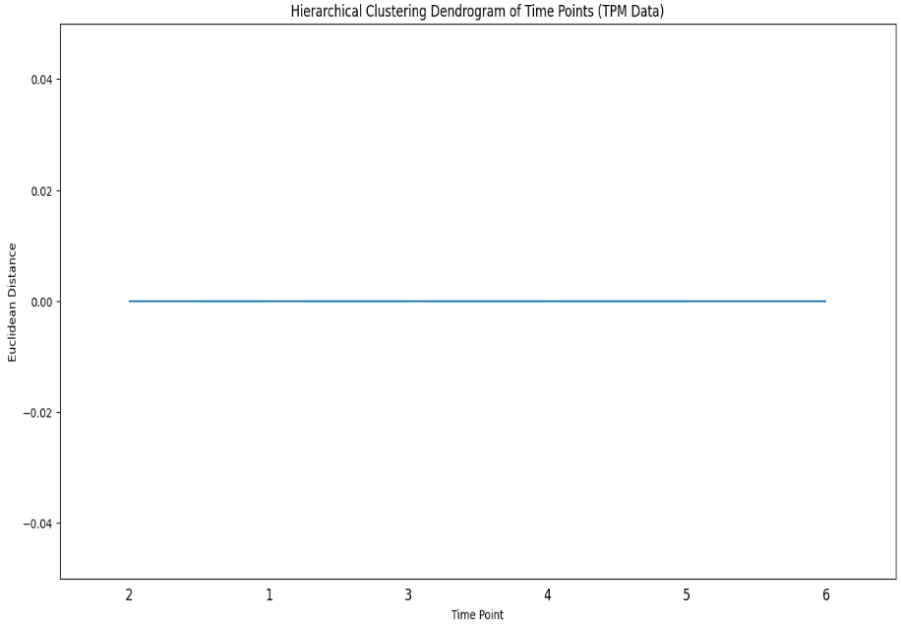


Figure 10. Hierarchical clustering dendrogram of timepoints by using ward's technique, depicting high similarity across all samples. Tight branching shows high similarity which confirms stable gene expression.

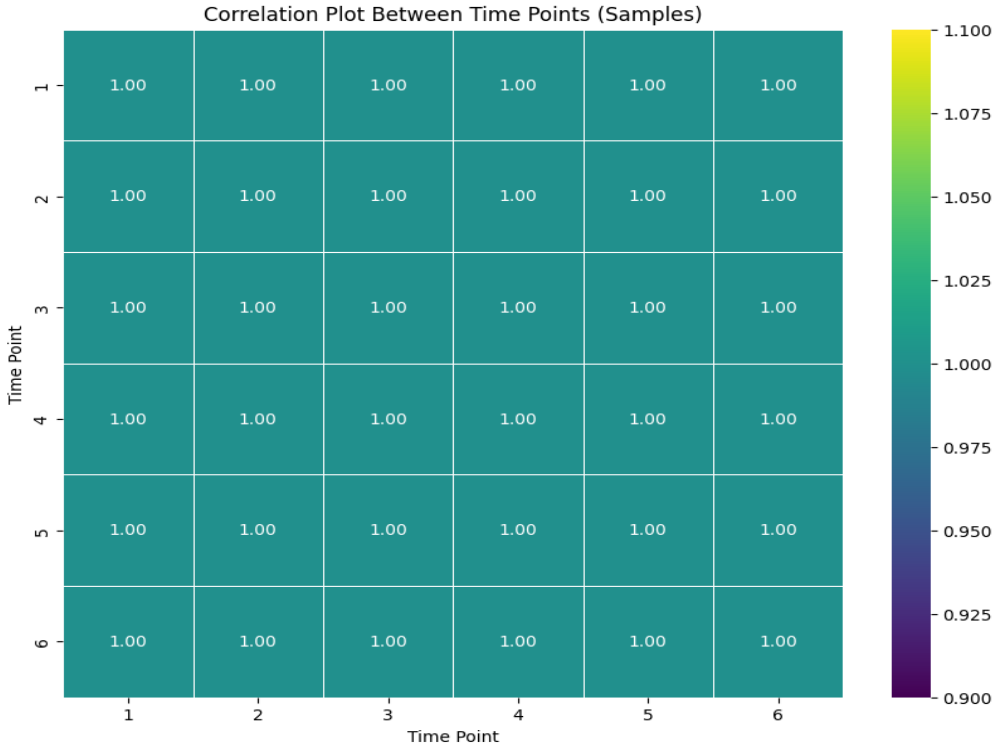


Figure 11. Correlation heatmap of time points displaying similarity in the correlation, reflecting nearly identical expression values.

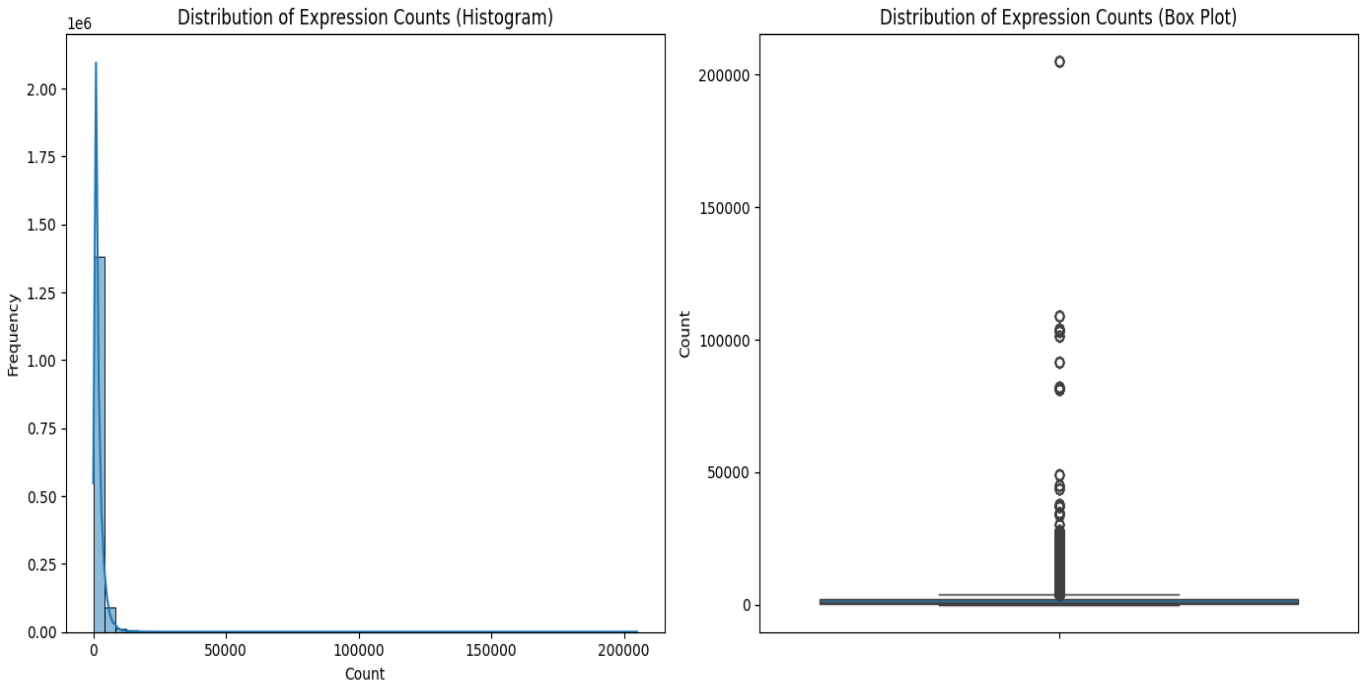
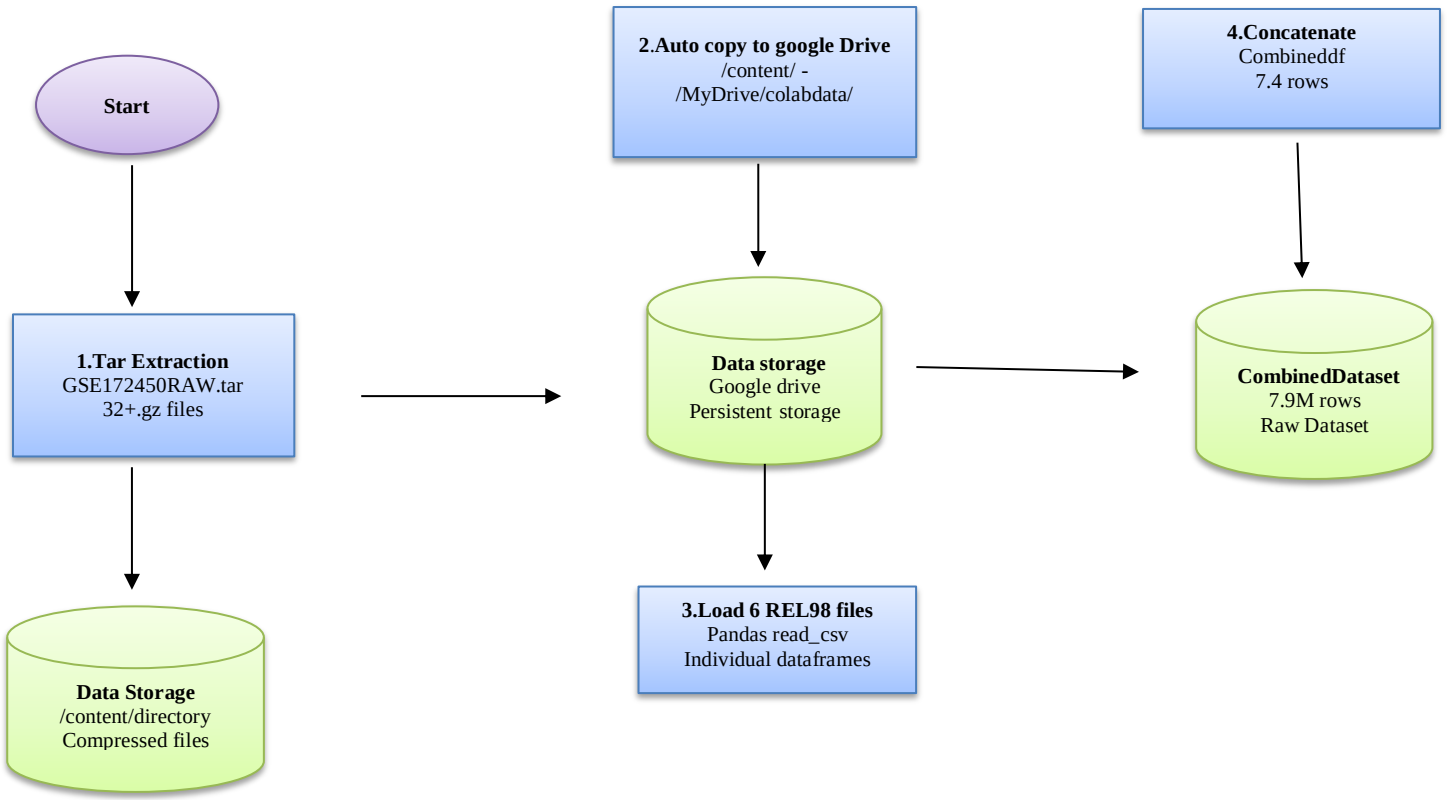
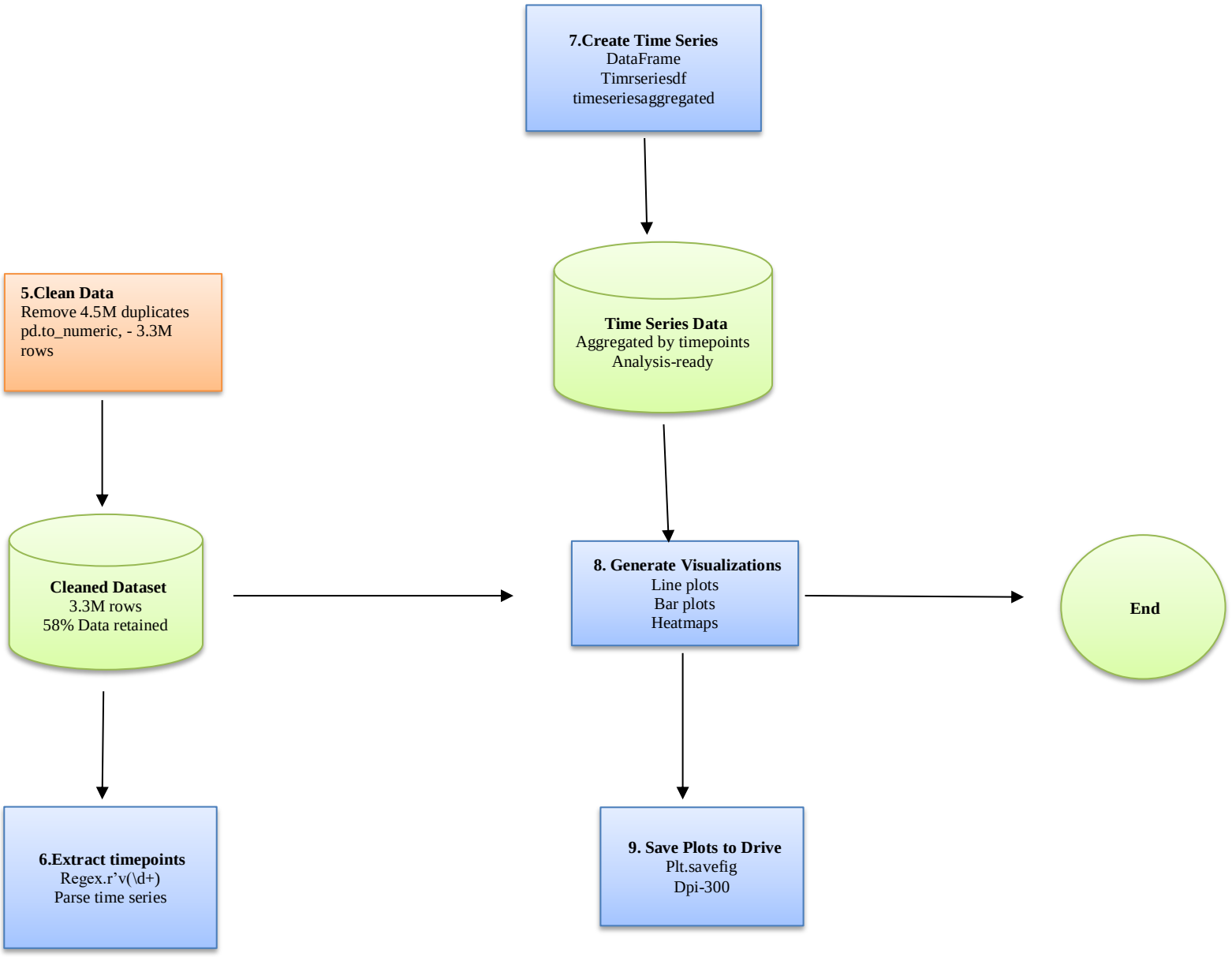


Figure 12. Histogram and Box plot of count distribution from timeseries, showing KDE curve with low-count genes and few high-expression.

Flowchart of data summary-





Note : All figures are generated using Matplotlib[4] plt.savefig(dpi = 300)

Results and Discussions

The time series analysis of gene expression data showed high stability across all examined time points. I successfully imported all six REL98 files after resolving initial file-path issues by converting them into the local/content/, resulting in an integrated time_series_df with time_point, information, gene identifiers, and count values. Data quality checks confirmed that there were no missing values, ensuring a well-organized dataset for futher analysis. The visualized methods such as line plots, bar charts, and heatmaps of the top 10 genes showed largely stable gene expression values over time, with little global changes. Gene expression normalization was performed by using CPM and TPM approach was applied, with fake gene lengths used to enabling RPK-to-TPM conversion. Thereafter TPM-based analyses showed very low variability across gene identifiers, supporting the stable expression trends. The additional Exploratory Data Analysis, including correlation plots, PCA, and hierarchical clustering, constantly showed high similarity among all time points. The correlations and tightly grouped clustered time points indicated that exceptional similarity in gene expression in the experiment. Differential expression analysis in the early and late stages failed to identified statistically significant expression changes, and gene clustering analysis showed common expression trend without clusters. Machine learning predictions models, which were using time_point + one-hot encoded identifiers as input features and y (TPM) values as outputs, reflected the largely static nature of gene expression dataset. The predictive results arrange themselves with earlier findings, capturing less variation in expression values. Overall, while this study successfully showed a detailed bioinformatics and predictive modeling techniques, the result suggest that the biological system is maintained transcriptional stability, due to inherent homeostasis regulation or limited temporal resolution to detect subtle gene expression changes.

Table 1 : Data Summary

Metric	Raw Value	Cleaned Value	Reduction
Total Rows	7,936,384	3,340,148	57.9%
Duplicate Rows	4,596,236	0	100%
Memory Usage	181.6 MB	101.9 MB	43.9%
Missing Values	Variable	0	100%

Table 2: Data cleaning operation

Operation	Code Used	Outcome
Duplicate Removal	drop_duplicates()	0 duplicates
Type Conversion	pd.to_numeric(errors='co erce')	Numeric format
Missing Value Removal	dropna()	Clean dataset
Drive Export	!cp *.gz /colabdata/	Persistent storage

Table 3: Visualization Status

Plot Type	**Status**	**Reason**
Bar Chart (Total Counts)	Success	timeseriesaggregatedavailable
Heatmap (Top Identifiers)	Failed	"timeseriesdf not available"
Time Series Line Plot	Not Implemented	Missing code

Conclusion

This report presents analysis of time-series gene expression data aimed at examining expression changes over time, identifying regulatory, and developing predictive models. The analysis followed an workflow that included careful loading and clean data, normalization by using CPM and TPM, exploratory data analysis (EDA), differential expression testing, gene clustering, and machine learning predictions. Across all stages of analyses, gene expression data were found to be highly stable, with very little changes over time. Aggregate expression values of Total counts remained constant, and heatmaps of highly expressed genes showed similar expression values across time-points. Correlation analysis showed high similarity between samples, while PCA failed to show clear separation in time-points. Hierarchical clustering further gave these findings by grouping all samples closely grouped together. Comparison between early and late time points did not find any strong changes in differential expression, and clustering gene expression also showed largely uniform trends without definite clusters. These results show that, within the studied experimental conditions and time frame, the biological system is markedly stable, or any changes are too small to detect with this available dataset. However, the google colab notebook successfully showcased a complete analysis workflow, applying all steps from data preprocessing and normalization to statistical analysis and predictive modelling. This work shows how time-series gene expression data can be effectively analysed even when the biological system shows variation over time is limited.

Future Prospects

For future research, the ability to detect better or cleaner potential changes in gene expression could be improve by examining experimental conditions that are known to produce strong transcriptional responses. This includes increasing the frequency or number of sampled time points to capture subtle or rapid changes, using more sensitive measurements techniques, focusing on specific pathways of interest's. In the overall observation, although the present dataset presents great stable gene expression across the examined time points, this report provides a complete and reproducible and adaptable framework for time-series analysis. The workflow showcased here can readily use in other datasets, specifically those with complex biological responses.

Reference

1. Python: Programming language for scientific computing (Python Software Foundation, Various versions) (https://www.python.org)
2. Pandas: Powerful Python data analysis toolkit (McKinney, 2010) (https://pandas.pydata.org)
3. NumPy: Fundamental package for numerical computing (Harris et al., 2020) (https://numpy.org)
4. Matplotlib: Python plotting library (Hunter, 2007) (https://matplotlib.org)
5. Seaborn: Statistical data visualization (Waskom, 2021) (https:/.pydata.org)
6. Scikit-learn: Machine learning in Python (Pedregosa et al., 2011) (https://scikit-learn.org)
7. XGBoost: Extreme Gradient Boosting (Chen & Guestrin, 2016) (https://xgboost.readthedocs.io)
8. SciPy: Scientific computing tools (Virtanen et al., 2020) (https://scipy.org)
9. TPM: Transcript-level expression normalization (Wagner et al., 2012)